

24k-0559

BAZIL – UDDIN –KHAN

BSCS 4H

Ai lab #2 TASK#1:

```
[5]: class Student:
      def __init__(self,name,rollno,program):
          self.name = name
          self.rollno = rollno
          self.program = program

      def display_info(self):
          print("Name is :",self.name)
          print("Roll No is :",self.rollno)
          print("Program is :",self.program)

s1 = Student("Ali","24K-0876","CS")
s2 = Student("nAFEY ","24K-0806","AI")
s3 = Student("QASIM","24K-0976","SE")
s1.display_info()
s2.display_info()
s3.display_info()
```

```
Name is : Ali
Roll No is : 24K-0876
Program is : CS
Name is : nAFEY
Roll No is : 24K-0806
Program is : AI
Name is : QASIM
Roll No is : 24K-0976
Program is : SE
```

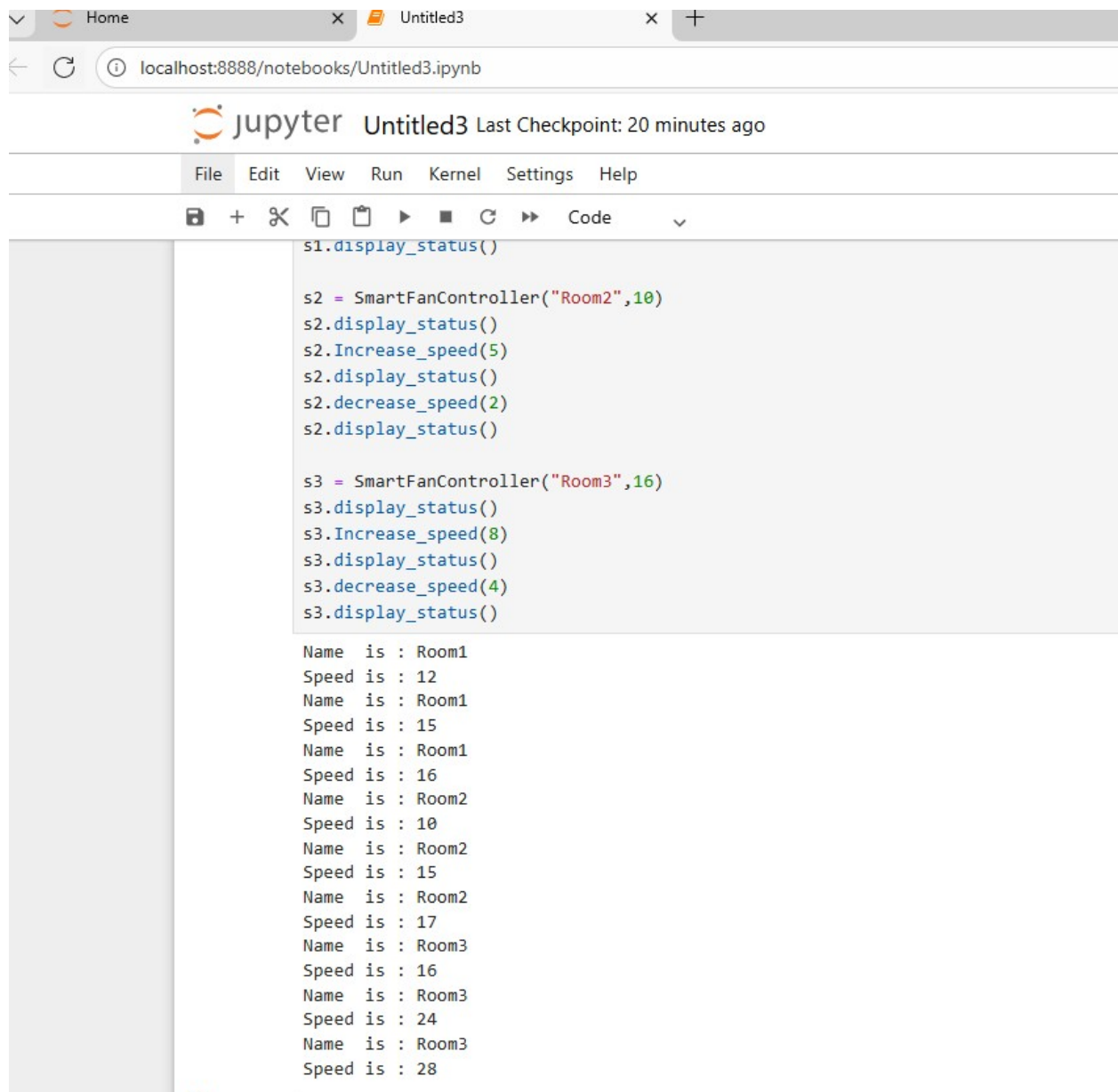
CODE:

```
class Student:
    def __init__(self, name, rollno, program):
        self.name = name
        self.rollno = rollno
        self.program = program

    def display_info(self):
        print("Name is :", self.name)
        print("Roll No is :", self.rollno)
        print("Program is :", self.program)

s1 = Student("Ali", "24K-0876", "CS")
s2 = Student("nAFEY", "24K-0806", "AI")
s3 = Student("QASIM", "24K-0976", "SE")
s1.display_info()
s2.display_info()
s3.display_info()
```

TASK#2:



The screenshot shows a Jupyter Notebook titled 'Untitled3' running on a local server at localhost:8888. The notebook contains two code cells. The first cell creates a SmartFanController object for 'Room1' with a speed of 12, then calls display_status(), increase_speed(5), display_status(), decrease_speed(2), and display_status(). The second cell creates a SmartFanController object for 'Room3' with a speed of 16, then calls display_status(), increase_speed(8), display_status(), decrease_speed(4), and display_status(). The output shows the status of Room1 (12, 15, 16) and Room2 (10, 15, 17) and Room3 (16, 24, 28).

```
s1.display_status()

s2 = SmartFanController("Room2",10)
s2.display_status()
s2.Increase_speed(5)
s2.display_status()
s2.decrease_speed(2)
s2.display_status()

s3 = SmartFanController("Room3",16)
s3.display_status()
s3.Increase_speed(8)
s3.display_status()
s3.decrease_speed(4)
s3.display_status()
```

Name is : Room1
Speed is : 12
Name is : Room1
Speed is : 15
Name is : Room1
Speed is : 16
Name is : Room2
Speed is : 10
Name is : Room2
Speed is : 15
Name is : Room2
Speed is : 17
Name is : Room3
Speed is : 16
Name is : Room3
Speed is : 24
Name is : Room3
Speed is : 28

Code:

```
class SmartFanController:    def
__init__(self,room_name,speed):
    self.room_name =room_name
    self.speed = speed
```

```
def Increase_speed(self,increase):  
self.speed = self.speed + increase
```

```
def decrease_speed(self,decrease):  
self.speed = self.speed - decrease
```

```
def display_status(self):  
print("Name is :",self.room_name)  
print("Speed is :",self.speed)
```

```
s1 = SmartFanController("Room1",12)  
s1.display_status()  
s1.Increase_speed(3)  
s1.display_status()  
s1.decrease_speed(1)  
s1.display_status()
```

```
s2 = SmartFanController("Room2",10)  
s2.display_status()  
s2.Increase_speed(5)  
s2.display_status()  
s2.decrease_speed(2)  
s2.display_status()
```

```
s3 = SmartFanController("Room3",16)
```

```
s3.display_status()
```

```
s3.Increase_speed(8)
```

```
s3.display_status()
```

```
s3.decrease_speed(4)
```

```
s3.display_status()
```

Task#3:

```
[12]: #TASK3
class Book:
    library_name = "City Central Library"

    def __init__(self, title, author):
        self.title = title
        self.author = author

    def display_book(self):
        print("NAME ", Book.library_name)
        print("TITLE  ", self.title)
        print("AUTHOR ", self.author)

book1 = Book("1984", "George Orwell")
book1.display_book()
Book.library_name = "National Digital Library"
book1.display_book()
```

```
NAME City Central Library
TITLE 1984
AUTHOR George Orwell
NAME National Digital Library
TITLE 1984
AUTHOR George Orwell
```

CODE:

```
class Book:
```

```
    library_name = "City Central Library"
```

```
    def __init__(self, title, author):
```

```
        self.title = title
```

```
    self.author = author    def
```

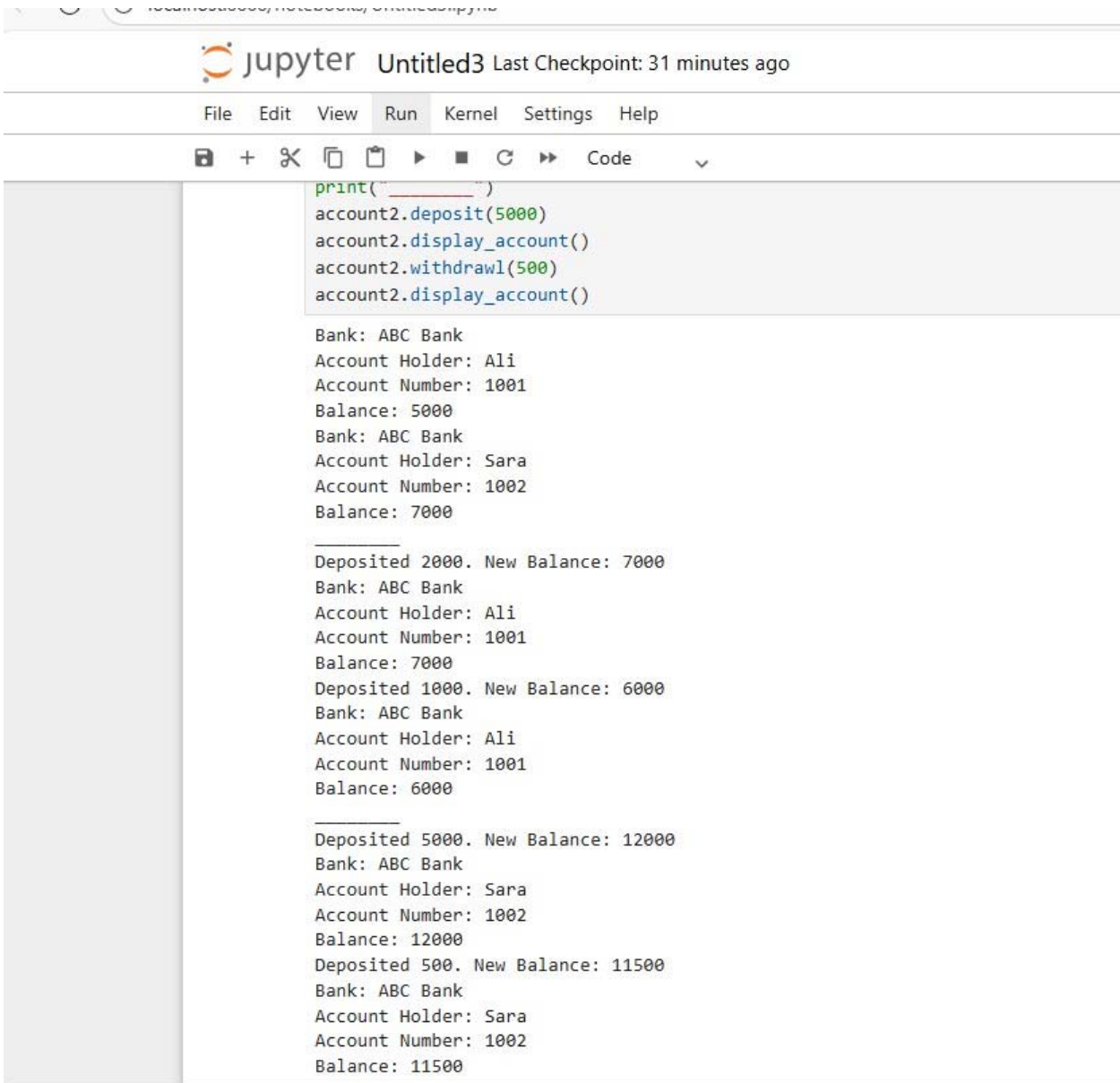
```
display_book(self):
```

```
    print("NAME
```

```
",Book.library_name)  
print("TITLE ",self.title)  
print("AUTHOR " ,self.author)
```

```
book1 = Book("1984", "George Orwell")  
book1.display_book()  
Book.library_name = "National Digital Library" book1.display_book()
```

task#4:



The image shows a Jupyter Notebook interface with a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The notebook is titled 'Untitled3' and shows the last checkpoint was 31 minutes ago. The code cell contains the following Python code:

```
print("\n")
account2.deposit(5000)
account2.display_account()
account2.withdrawl(500)
account2.display_account()
```

The output of the code is as follows:

```
Bank: ABC Bank
Account Holder: Ali
Account Number: 1001
Balance: 5000
Bank: ABC Bank
Account Holder: Sara
Account Number: 1002
Balance: 7000

Deposited 2000. New Balance: 7000
Bank: ABC Bank
Account Holder: Ali
Account Number: 1001
Balance: 7000
Deposited 1000. New Balance: 6000
Bank: ABC Bank
Account Holder: Ali
Account Number: 1001
Balance: 6000

Deposited 5000. New Balance: 12000
Bank: ABC Bank
Account Holder: Sara
Account Number: 1002
Balance: 12000
Deposited 500. New Balance: 11500
Bank: ABC Bank
Account Holder: Sara
Account Number: 1002
Balance: 11500
```

CODE:

class BankAccount:

```
    bank_name = "ABC Bank"    def __init__(self,
holder_name, account_no, balance):
```

```
        self.holder_name = holder_name
self.account_no = account_no
self.balance = balance
```

```
def display_account(self):
    print("Bank:", BankAccount.bank_name)
print("Account Holder:", self.holder_name)
print("Account Number:", self.account_no)
print("Balance:", self.balance)
```

```
def deposit(self, amount):
self.balance += amount
    print(f"Deposited {amount}. New Balance: {self.balance}")
```

```
def withdrawl(self, amount):
self.balance -= amount
    print(f"Deposited {amount}. New Balance: {self.balance}")
```

```
account1 = BankAccount("Ali", 1001, 5000)
account2 = BankAccount("Sara", 1002, 7000)
account1.display_account()
account2.display_account()
print("_____") account1.deposit(2000)
account1.display_account()
```

```
account1.withdrawl(1000)
account1.display_account()
print("_____") account2.deposit(5000)
account2.display_account()
account2.withdrawl(500)
account2.display_account()
```

task#5:

```
l8]: #Task5
```

```
class SmartLight:

    def __init__(self,name,status):
        self.name = name
        self.status = status

    def DisplayOn(self):
        print(" " , self.name , " Light is " ,self.status)

    def DisplayOff(self):
        print(" " , self.name , " Light is " ,self.status)

s1 = SmartLight("Bed room","On")
s2 = SmartLight("Living room","On")
s3 = SmartLight("Bed room","Off")
s4 = SmartLight("living room","Off")

s1.DisplayOn()
s2.DisplayOff()
s3.DisplayOn()
s4.DisplayOff()
```

```
Bed room Light is On
Living room Light is On
Bed room Light is Off
living room Light is Off
```

CODE:

```
class SmartLight:
```

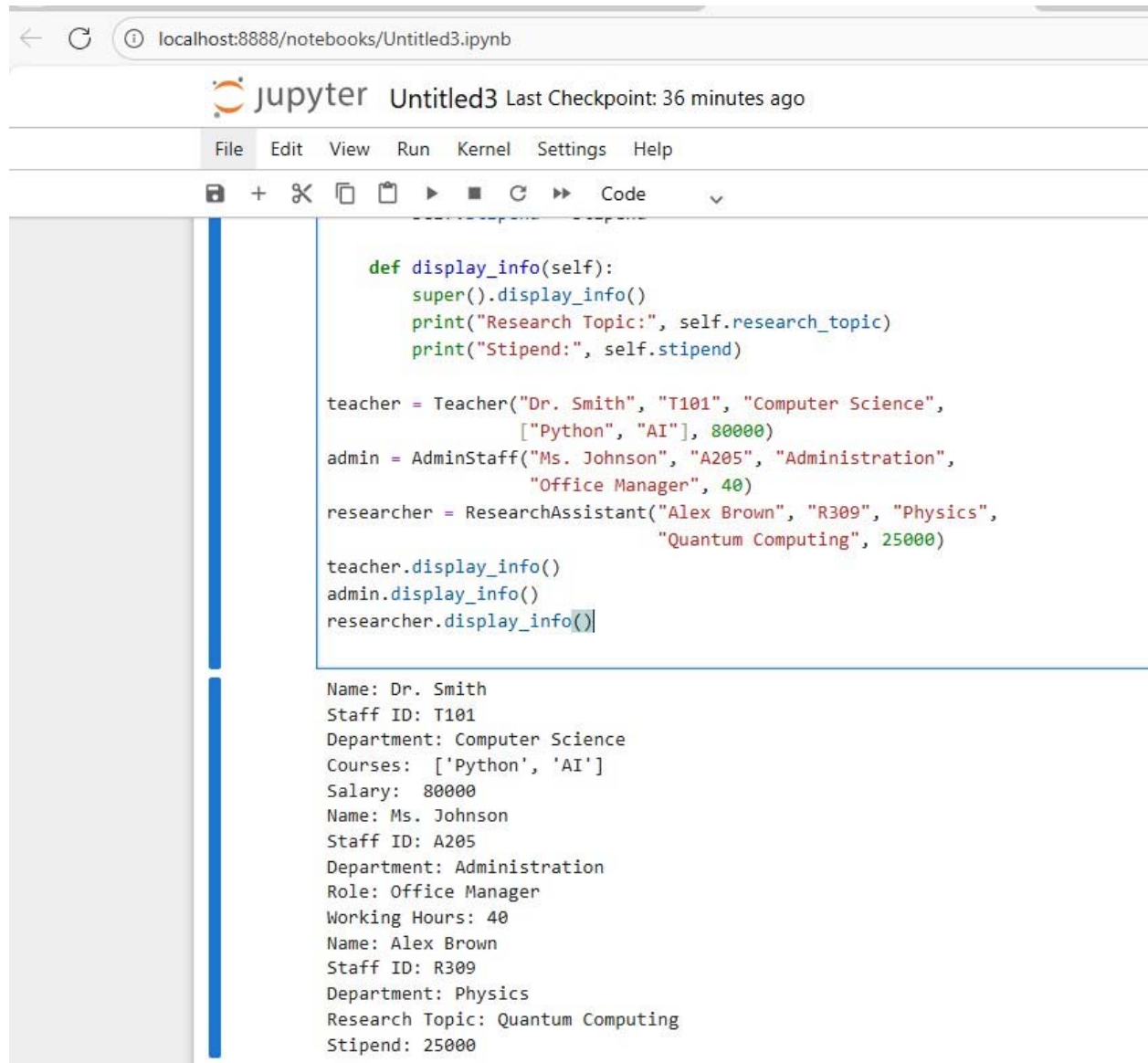
```
def __init__(self,name,status):
    self.name = name
self.status = status

def DisplayOn(self):
    print(" ", self.name , " Light is " ,self.status)

def DisplayOff(self):
    print(" ", self.name , " Light is " ,self.status)

s1 = SmartLight("Bed room","On") s2
= SmartLight("Living room","On") s3
= SmartLight("Bed room","Off") s4 =
SmartLight("living room","Off")
s1.DisplayOn() s2.DisplayOff()
s3.DisplayOn() s4.DisplayOff()
```

task#6:



The screenshot shows a Jupyter Notebook running in a web browser at localhost:8888. The notebook is titled 'Untitled3' and shows the last checkpoint from 36 minutes ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for saving, adding, deleting, and running code. The code cell contains a Python script that defines a base class 'Staff' and three subclasses: 'Teacher', 'AdminStaff', and 'ResearchAssistant'. Each subclass has a 'display_info' method. The script then creates instances of these classes and calls their 'display_info' methods. The output cell shows the results of these calls, displaying attributes like Name, Staff ID, Department, Courses, Salary, Role, and Research Topic for each instance.

```
def display_info(self):
    super().display_info()
    print("Research Topic:", self.research_topic)
    print("Stipend:", self.stipend)

teacher = Teacher("Dr. Smith", "T101", "Computer Science",
                  ["Python", "AI"], 80000)
admin = AdminStaff("Ms. Johnson", "A205", "Administration",
                  "Office Manager", 40)
researcher = ResearchAssistant("Alex Brown", "R309", "Physics",
                              "Quantum Computing", 25000)

teacher.display_info()
admin.display_info()
researcher.display_info()
```

Name: Dr. Smith
Staff ID: T101
Department: Computer Science
Courses: ['Python', 'AI']
Salary: 80000
Name: Ms. Johnson
Staff ID: A205
Department: Administration
Role: Office Manager
Working Hours: 40
Name: Alex Brown
Staff ID: R309
Department: Physics
Research Topic: Quantum Computing
Stipend: 25000

CODE:

```
class Staff:
    def __init__(self, name, staff_id,
department):
        self.name = name
        self.staff_id = staff_id
```

```
self.department = department
def display_info(self):
    print("Name:",self.name)
    print("Staff ID:", self.staff_id)
    print("Department:",self.department)
```

```
class Teacher(Staff):
    def __init__(self, name, staff_id, department, courses, salary):
        super().__init__(name, staff_id, department)
    self.courses = courses    self.salary = salary
```

```
    def display_info(self):
    super().display_info()
    print("Courses: ",self.courses)
    print("Salary: ",self.salary)
```

```
class AdminStaff(Staff):    def __init__(self, name, staff_id,
department, role, working_hours):
    super().__init__(name, staff_id, department)
    self.role = role
    self.working_hours = working_hours
```

```

    def display_info(self):
super().display_info()    print("Role:",
self.role)

    print("Working Hours:",self.working_hours) class
ResearchAssistant(Staff):    def __init__(self, name, staff_id, department,
research_topic, stipend):
    super().__init__(name, staff_id, department)
self.research_topic = research_topic    self.stipend
= stipend

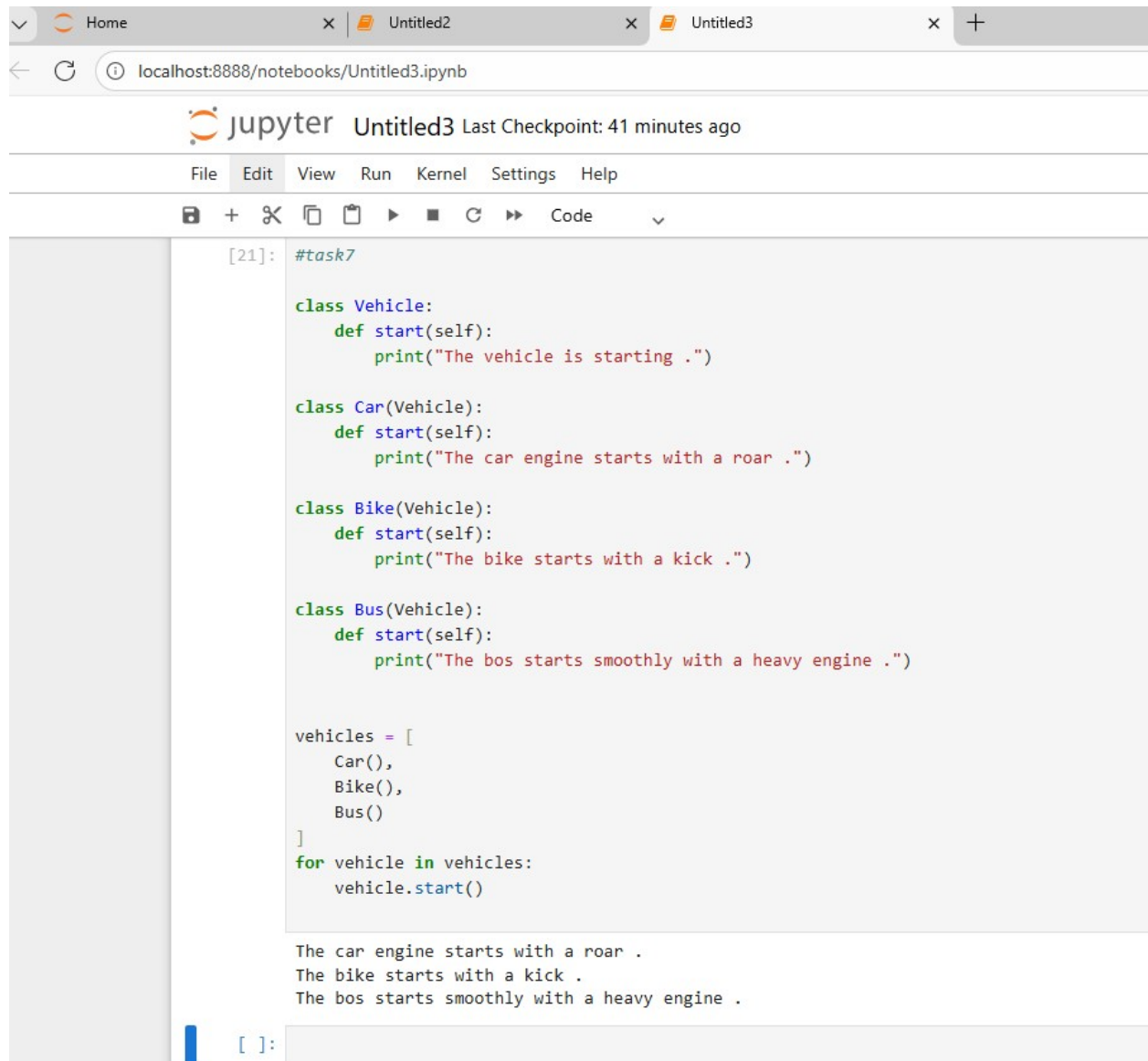
    def display_info(self):
super().display_info()

    print("Research Topic:", self.research_topic)
print("Stipend:", self.stipend)

teacher = Teacher("Dr. Smith", "T101", "Computer Science",
    ["Python", "AI"], 80000)
admin = AdminStaff("Ms. Johnson", "A205", "Administration",
    "Office Manager", 40)
researcher = ResearchAssistant("Alex Brown", "R309", "Physics",
    "Quantum Computing", 25000)
teacher.display_info() admin.display_info()
researcher.display_info()

```

task#7:



The screenshot shows a Jupyter Notebook titled 'Untitled3' running on a local server at localhost:8888. The notebook contains a Python script defining a base class 'Vehicle' and three subclasses: 'Car', 'Bike', and 'Bus'. Each class has a 'start' method that prints a specific message. A list named 'vehicles' contains instances of these classes, and a loop iterates over the list, calling the 'start' method for each instance. The output of the code is displayed below the code cell, showing the three print statements.

```
[21]: #task7

class Vehicle:
    def start(self):
        print("The vehicle is starting .")

class Car(Vehicle):
    def start(self):
        print("The car engine starts with a roar .")

class Bike(Vehicle):
    def start(self):
        print("The bike starts with a kick .")

class Bus(Vehicle):
    def start(self):
        print("The bos starts smoothly with a heavy engine .")

vehicles = [
    Car(),
    Bike(),
    Bus()
]
for vehicle in vehicles:
    vehicle.start()

The car engine starts with a roar .
The bike starts with a kick .
The bos starts smoothly with a heavy engine .
```

CODE:

```
class Vehicle:
```

```
    def start(self):
```

```
        print("The vehicle is starting .")
```

```
class Car(Vehicle):
```

```
def start(self):  
    print("The car engine starts with a roar .")
```

```
class Bike(Vehicle):  
    def start(self):  
        print("The bike starts with a kick .")
```

```
class Bus(Vehicle):  
    def start(self):  
        print("The bos starts smoothly with a heavy engine .")
```

```
vehicles = [  
    Car(),  
    Bike(),  
    Bus()  
]  
for vehicle in vehicles:  
    vehicle.start()
```

TASK#8:

```

•[25]: #task 8
class Calculator:
    def multiply(self, *numbers):
        result = 1
        for x in range(numbers):
            result *= num
        return result

    def multiply(self, a, b, c=1):
        return a * b * c

c1 = Calculator()
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
c1 = Calculator()
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
print(calc.multiply(1, 2, 3))

```

```

6
24
6
24
6

```

CODE: class Calculator: def

multiply(self, *numbers):

result = 1 for x in

range(numbers):

result *= num

return result

def multiply(self, a, b, c=1):

return a * b * c

```
c1 = Calculator()
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))  c1
= Calculator()
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
print(calc.multiply(1, 2, 3))
```

task#9:

```
[26]: # task#9
class Employee:
    def __init__(self, name, salary):
        self._name = name
        self._salary = salary

class Manager(Employee):
    def display_info(self):
        print("Manager Name:", self._name)
        print("Manager Salary:", self._salary)

manager = Manager("Alice Johnson", 75000)
manager.display_info()
```

```
Manager Name: Alice Johnson
Manager Salary: 75000
```

```
[ ]: |
```

Code:

```
class Employee:
    def __init__(self, name, salary):
        self._name = name
        self._salary = salary

class Manager(Employee):
    def display_info(self):
        print("Manager Name:",self._name)
        print("Manager Salary:",self._salary)

manager = Manager("Alice Johnson", 75000) manager.display_info()
```

task#10:

```
[27]: #TASK 10
class BankAccount:
    def __init__(self, initial_balance=0):
        self.__balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Deposited amount ")
        else:
            print("Invalid cant deposit amount ")

    def withdraw(self, amount):
        if amount > 0 and amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrew {amount}")
        else:
            print("Insufficient funds or invalid amount.")

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
print(account.balance)
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[27], line 25
     21         return self.__balance
     24 account = BankAccount(1000)
--> 25 print(account.balance)
```

code:

```
class BankAccount:
    def
    __init__(self, initial_balance=0):
        self.__balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
```

```
        print("Deposited amount ")
else:
    print("Invalid cant deposit amount ")

    def withdraw(self, amount):
        if amount > 0
and amount <= self.__balance:
        self.__balance -= amount
    print(f"Withdrew {amount}")
    else:
        print("Insufficient funds or invalid amount.")

    def get_balance(self):
return self.__balance

account = BankAccount(1000)
print(account.balance)
```